Unix processes have three channels for I/O: standard input (stdin), standard output (stdout), and standard error (stderr). Since the Docker daemon runs in a separate process, Docker would have to actively do something to forward our CLI input to the Docker daemon.

By default, however, docker run only forwards the container's output to our client. That's fine when we want to run a container that requires no input. However, sometimes we run processes that do require input. An interactive Bash session is a great example of this—it waits to receive the commands we enter. In this case, we need to explicitly tell Docker to forward our CLI input on to the Docker daemon. We do this with the -i option—"*i*" for input. If we didn't specify this, the container would terminate immediately, because Bash—receiving no input —would terminate.

However, this alone is not enough. An interactive Bash session must be run inside a *terminal emulator*, which is responsible for things like displaying a prompt and interpreting escape sequences such as Ctrl-C. If we start a container to run bash, by default this runs in noninteractive mode, executing any commands provided, and terminating once it's done. To achieve a long-lived, interactive Bash session inside a Docker container, we have to tell Docker to set up a terminal emulator for us (technically a pseudoterminal or pty) that sits in front of Bash. We do this by specifying the -t option for docker run.

Now, if this all sounds quite complicated, just remember that whenever you need a long-lived, interactive session, you need to specify both the -i and -t options. In fact, these are commonly combined into the shorthand form -it, which you can think of as meaning "i"-n-"t"-eractive. Neat.

And with that, our work here is done.

Quick Recap

Now that you've had your first taste of Docker, let's pause for a moment to catch our breath and review what we've learned.

In this chapter:

1. We installed Docker on our machines.
2. We ran our first ever Docker command, a *helloworld* Ruby script, without needing Ruby installed on our machine.

   ```
   $ docker run ruby:2.6 ruby -e "puts :hello"
   ```

3. We saw how to list our running containers with docker ps and all containers (including stopped ones) with docker ps -a.